CADENCE: Case-Grounded, Deliberation-Driven, and Solver-Verified Architectural Decision-Making with Large Language Models

Quang-Vinh Dang

British University Vietnam, Hung Yen, Vietnam

Abstract

Context: Large language models (LLMs) are increasingly used to draft architecture decision records (ADRs). Existing LLM approaches do not check whether a decision addresses the quality attributes at stake within what a team can build. Architecture optimization checks decisions, but only on hand-specified design spaces. **Objective:** We join the two lines of work and identify which parts of an LLM decision pipeline improve the decision and where it fails. **Method:** CADENCE retrieves precedent ADRs from 6,173 mined records, lets five agents each advocate one ISO/IEC 25010 quality attribute grounded in a tactic knowledge graph, encodes the decision's tactic commitments as a lexicographic weighted MaxSAT problem in Z3 with an LLM repair loop, and scores the result. We evaluate it on 30 held-out open-source ADRs in a paired ablation that adds one stage at a time with a local 1.5B-parameter backbone, two tactic budgets, Holm-corrected Wilcoxon tests, a length control, and published frontier-model generations. **Results:** Retrieval slightly improves agreement with the recorded decisions. Multi-agent deliberation lowers it on all four metrics, by more than its shorter output explains. The solver correctly flags all 30 deliberated decisions as leaving required quality attributes without a committed tactic. Repair raises mean coverage from 0.57 to 1.63 of five attributes but makes only one decision feasible; extra budget does not help, because the committed tactics cluster on too few attributes. The critique stage does not change the decision text, and an earlier small-sample gain attributed to it was sampling noise. Frontier

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

models agree more with the recorded ADRs than every CADENCE variant. **Conclusion:** Formal verification reliably detects quality-attribute gaps in LLM-drafted decisions, but post-hoc repair is not enough. Decisions should commit to tactics in structured form, and evaluation should go beyond agreement with a single recorded ADR.

Keywords: Software architecture, architectural decision-making, architecture decision records, architectural tactics, large language models, constraint solving, empirical evaluation

---

1. Introduction

Architectural decisions are the design choices that are expensive to reverse: introducing a message queue, splitting a service, adopting event sourcing, choosing where encryption terminates. Software architecture research has treated these decisions, and the rationale behind them, as first-class architectural knowledge for two decades (Jansen and Bosch, 2005; Kruchten, 2004; Tang et al., 2006), and practitioners record them as architecture decision records (ADRs) (Tyree and Akerman, 2005), a practice now visible in hundreds of open-source repositories (Buchgeher et al., 2023). Empirical studies of how architects actually decide report a consistent picture. Decisions are made under time pressure with partial knowledge of precedent (van Vliet and Tang, 2016; Weinreich et al., 2015); trade-offs among quality attributes are weighed informally and often implicitly (Zalewski et al., 2017; Falessi et al., 2011); and the process is exposed to cognitive biases such as anchoring on the first option considered (Borowa et al., 2021; Razavian et al., 2016). Systematic methods exist to counter this — ATAM and CBAM for trade-off analysis (Kazman et al., 1998, 2001), tactic catalogs linking design choices to quality attributes (Bachmann et al., 2003; Bass et al., 2021), and search-based architecture optimization (Aleti et al., 2013; Koziolek et al., 2011) — but they demand expert effort or a formal architecture model that is rarely available when a decision is first drafted.

Large language models (LLMs) lower that effort, and the software architecture community has started to use them for exactly this task: drafting ADRs from a decision context (Dhar et al., 2024), grounding the draft in mined precedent (Gupta et al., 2026b), splitting architecture design across role-based agents (Zhang et al., 2025), and supporting quality-attribute or ATAM-style reasoning (Diaz-Pace et al., 2025; Capilla et al., 2025). Recent

surveys and roadmaps record the same open problem across this work (Hou et al., 2024; Esposito et al., 2026; Schmid et al., 2025): an LLM's decision is text, and nothing in these pipelines checks that the tactics it commits to actually cover the quality attributes at stake, or that they fit within what the team can build. In MAAD, for instance, the final evaluation step is another LLM call (Zhang et al., 2025). Conversely, the architecture-optimization tradition offers exactly that kind of check, but expects its design space to be specified by hand (Aleti et al., 2013; Martens et al., 2010). The two lines have not been joined.

This paper joins them. We present CADENCE, an approach to LLM-assisted architectural decision-making that treats the LLM as a generator of decision alternatives and rationale, and treats the decision's tactic commitments as a design-space problem to be checked formally. CADENCE has four stages. (1) It retrieves the closest precedent ADRs from a corpus of 6,173 mined records (Buchgeher et al., 2023). (2) Five agents, each advocating one ISO/IEC 25010-derived quality attribute (ISO/IEC, 2023) and grounded in a 26-tactic knowledge graph with explicit trade-off edges, deliberate over a candidate decision, so that competing concerns are argued for rather than averaged away. (3) The tactics the candidate commits to are encoded as a lexicographic weighted MaxSAT problem in the Z3 SMT solver: cover every required quality attribute within a tactic budget B, then minimize trade-off cost. If the encoding is unsatisfiable, the uncovered attributes drive an LLM repair loop. (4) A finalization pass combines a deterministic utility score, computed from the solver's selection and the graph's trade-off edges, with an LLM critique of residual weaknesses.

We evaluate CADENCE as one would evaluate a decision-support tool for software architects, on held-out human-authored ADRs and with an ablation in which each system adds one stage to the previous one. The evaluation addresses four research questions:

RQ1 (stage contribution)
How does each stage — precedent retrieval, quality-attribute deliberation, solver verification, and critique — change agreement with the decisions architects actually recorded?

RQ2 (feasibility)
Can solver verification with repair make an LLM-proposed decision cover all required quality attributes within a tactic budget, and how often does it succeed as the budget gains slack?

RQ3 (external reference)
How do CADENCE's decisions compare with ADRs generated by frontier LLMs on the same items, as published in an independent replication package (Gupta et al., 2026a)?

RQ4 (measurement)
How well do reference-based text metrics, the standard instrument in LLM-for-ADR studies, capture differences between structured, multi-stage decision output and free-form generations?

The paper makes four contributions to software architecture research.

1. *An approach that couples LLM-generated decision alternatives with formal design-space verification.* CADENCE links the LLM-for-ADR line of work to tactic-based reasoning (Bachmann et al., 2003; Bass et al., 2021) and to solver-based architecture optimization (Aleti et al., 2013). To our knowledge it is the first ADR-generation approach in which a decision's quality-attribute coverage is checked by a solver rather than judged by another LLM (Table 1).

2. *An encoding from free-text deliberation to a budgeted coverage problem.* We show that the encoding must be lexicographic: under a weighted sum, a large trade-off weight can outvote coverage, and a decision that leaves an attribute uncovered is then reported as optimal (Section 3.4).

3. *A paired ablation study on held-out ADRs from open-source projects.* From the deliberation onward, all systems share the same intermediate outputs, so each comparison isolates exactly one stage. The study shows that the solver reliably detects decisions that leave quality attributes uncovered (all 30 deliberated decisions), and that the binding constraint is the diversity of the tactics a decision commits to, not the tactic budget. We release the per-item outputs and solver diagnostics.

4. *Two lessons about measurement.* First, reference-based metrics penalize CADENCE's output beyond what its shorter length explains. Second, ablations of stochastic pipelines that re-sample shared stages can produce spurious stage effects. Our own earlier small-sample result for the critique stage was one such effect. Both lessons bear on how LLM-for-architecture studies should report results (Evtikhiev et al., 2023; Roy et al., 2021).

Section 2 positions CADENCE in the architecture-knowledge, decision-making, optimization and LLM-for-architecture literature. Section 3 presents the approach. Section 4 reports the evaluation by research question, Section 5 discusses implications for practice and research and the threats to validity, and Section 6 concludes.

2. Related Work

CADENCE draws on six bodies of software engineering research: the view of architecture as a set of design decisions and the management of the knowledge behind them; empirical work on how architects decide; trade-off analysis and quantitative selection methods; automated architecture optimization; architectural tactics and their quality-attribute relationships; and the recent use of large language models in software architecture. We review each and close the section with a summary of how CADENCE differs from the closest prior work (Table 1).

2.1. Architectural decisions and architecture knowledge

Jansen and Bosch (2005) argued that a software architecture is best understood as a set of architectural design decisions, and that losing those decisions and their rationale, rather than losing the structure itself, is what makes architectures hard to evolve. Tyree and Akerman (2005) gave the decision record its practical form, with fields for the issue, assumptions, alternatives, argument and implications that today's ADR templates still carry, and Kruchten (2004) classified decisions (existence, property and executive decisions) and the relations between them, such as constrains, forbids and conflicts with. Zimmermann et al. (2009) made such relations operational: their reusable decision models carry dependency relations and formal integrity constraints, so that a tool can check whether a set of decisions is consistent. CADENCE inherits this idea of constraint-checked decisions. Its constraints are narrower (a tactic budget and quality-attribute coverage), but they are checked by a solver against an explicit tactic model rather than by rules over a decision tree, and the decisions themselves are proposed by language-model agents.

The architecture knowledge management (AKM) literature documents why this matters in practice. Practitioners value design rationale but seldom record it (Tang et al., 2006); AKM tools support capture, sharing and reasoning to different degrees (Tang et al., 2010); and a ten-year retrospective

names the cost of capture and the weak reuse of captured knowledge as the main obstacles to adoption (Capilla et al., 2016). Open-source projects now publish ADRs in large numbers, although adoption per repository remains sparse (Buchgeher et al., 2023). CADENCE treats these mined ADRs as reusable knowledge: Stage 1 retrieves them as precedents for a new decision, and Stages 2–4 record the rationale (agent positions, solver verdict, repair history, critique) as a by-product of making the decision.

2.2. Architectural decision-making in practice

Architects rarely optimize. They satisfice under bounded rationality, decide in groups, and are subject to cognitive biases (van Vliet and Tang, 2016). Anchoring and confirmation bias have been observed in architecture decisions (Zalewski et al., 2017) and linked to architectural technical debt (Borowa et al., 2021), while prompting designers to reflect on their reasoning improves design decisions (Razavian et al., 2016). An expert survey reports which kinds of decisions and which influence factors, including cost and team expertise, dominate in practice (Weinreich et al., 2015). A comparative survey of decision-making techniques for architecture design describes the assumptions behind weighted scoring, AHP and related multi-criteria methods (Falessi et al., 2011). These findings shape three design choices in CADENCE. The adversarial quality-attribute advocates are a structured form of group deliberation intended to keep any single viewpoint from dominating; the separate critique pass is a form of reflection; and the lexicographic objective in Stage 3 is a deliberate choice among the multi-criteria techniques that Falessi et al. (2011) compare, made so that coverage of a required attribute can never be traded away for a lower trade-off cost (Section 3.4).

2.3. Trade-off analysis and quantitative selection

The Architecture Tradeoff Analysis Method (ATAM) organizes an evaluation around a quality-attribute utility tree and identifies sensitivity points and trade-off points in a candidate architecture (Kazman et al., 1998). The Cost Benefit Analysis Method (CBAM) extends it by choosing among architectural strategies according to their benefit per unit cost under a budget (Kazman et al., 2001); experience with CBAM showed that eliciting reliable utilities from stakeholders is the hardest step (Moore et al., 2003). Al-Naeem et al. (2005) combined multi-criteria scoring with integer programming to select design alternatives under quality and resource constraints. CADENCE's Stage 3 belongs to this tradition: it, too, selects architectural

strategies under a budget so as to satisfy quality-attribute requirements. What differs is where the inputs come from. CBAM and Al-Naeem et al. (2005) assume that stakeholders enumerate the alternatives and supply the scores; CADENCE derives candidate tactics from a natural-language decision context through retrieval and deliberation, scores trade-offs from an explicit tactic model, and repairs a candidate automatically when the selection is infeasible. It is meant to support such methods, not to replace a stakeholder-driven evaluation.

2.4. Automated architecture optimization

A systematic review of software architecture optimization classifies approaches by problem, degrees of freedom, quality evaluation and optimization strategy, and finds that most rely on quantitative quality models and metaheuristic search (Aleti et al., 2013). Representative tools search over component and deployment models for performance, reliability and cost (Martens et al., 2010; Aleti et al., 2009); PerOpteryx applies architectural tactics as search operators in multi-objective optimization (Koziolek et al., 2011); and recent work explores many-objective refactoring of software models (Cortellessa et al., 2023). Solver-based synthesis of architectures from formal style specifications (Bagheri and Sullivan, 2016) and the SAT/CSP/SMT analysis of feature models, including optimal configuration under cost constraints (Benavides et al., 2010), show that constraint solvers are well established for architecture-level choices. In the terms of Aleti et al. (2013), CADENCE's degree of freedom is the selection of tactics, its quality evaluation is a qualitative tactic model plus an LLM critique, and its strategy is exact (weighted MaxSAT/SMT) rather than metaheuristic. It does not need a component model or performance predictions, which makes it applicable earlier than model-based optimizers, but its quality evaluation is correspondingly coarser. PerOpteryx in particular already automates tactic application across several quality attributes; CADENCE differs in starting from natural-language context and precedents instead of an architecture model, and in using a solver to verify and repair an LLM-proposed decision rather than to search a model-defined design space.

2.5. Architectural tactics and quality attributes

Tactics were introduced as design primitives derived from quality-attribute reasoning frameworks (Bachmann et al., 2003) and are catalogued in Bass et al. (2021). Tactics interact with architecture patterns and with

one another, sometimes with negative effects on other attributes (Harrison and Avgeriou, 2010; Harrison et al., 2007); such relationships have been mined from developer discussions (Bi et al., 2021), detected and traced in source code (Mirakhorli et al., 2012; Mirakhorli and Cleland-Huang, 2016), mapped systematically (Márquez et al., 2023), and catalogued for specific system classes together with their trade-offs (Indykov et al., 2025). CADENCE's knowledge graph encodes one such catalog: 26 tactics, each with a *supports* edge to one ISO/IEC 25010-derived attribute (ISO/IEC, 2023) and trade-off edges to the attributes it burdens (Appendix A). The graph is hand-curated and deliberately small; we discuss how it could be grounded in mined tactic knowledge such as Bi et al. (2021) in Section 5.

2.6. Large language models in software architecture

Surveys of LLMs for software engineering find comparatively little work on design and architecture (Hou et al., 2024), call for combining LLMs with established software engineering techniques to contain hallucination (Fan et al., 2023), and describe the role-specialized agent organizations used in LLM-based multi-agent systems (He et al., 2025; Liu et al., 2026). Reviews specific to architecture report that most primary studies use direct prompting (Schmid et al., 2025) and list reliability, evaluation and rigour among the open challenges (Esposito et al., 2026); practitioner reports and early case studies show interest alongside caution (Jahić and Sami, 2024; Ahmad et al., 2023). An exploratory study found that the architectural knowledge contained in a general-purpose model is incomplete and sometimes wrong (Soliman and Keim, 2025), which motivates grounding in explicit knowledge.

For decisions specifically, Dhar et al. (2024) showed that a single LLM can draft a plausible ADR from a stated problem, follow-on work treats ADR authoring as an iterative drafting process (Dhar et al., 2025), and retrieval of context and precedent improves ADR generation (Gupta et al., 2026b,a). Related work generates the rationale for decisions already made (Zhou et al., 2025), detects violations of recorded decisions (Su et al., 2026), and uses agents to recover ADRs from repositories (Dhar et al., 2026); these recover or check decisions after the fact rather than make a new one under constraints. Three systems are closest to CADENCE. MAAD organizes knowledge-grounded agents into a pipeline of functional roles (analyst, modeler, designer, evaluator) that designs an architecture from requirements, with an LLM evaluator assessing quality attributes (Zhang et al., 2025; Li et al., 2025). Diaz-Pace et al. (2025) let LLM-based design agents explore

patterns and tactics for given requirements and reflect on the pros and cons of their decisions. Capilla et al. (2025, 2026) use LLM tools to analyse risks, sensitivity points and trade-offs in ATAM scenarios. All three rely on language models to judge quality attributes and feasibility. CADENCE differs in three ways: its agents are advocates for competing quality attributes that debate one decision, not stages of a pipeline; a solver, not a model, decides whether the decision fits the tactic budget and covers every required attribute, and drives repair when it does not; and deliberation starts from precedents retrieved from mined ADRs.

Outside architecture, LLMs have been paired with symbolic solvers that take over the deductive step (Pan et al., 2023; Olausson et al., 2023; Szeider, 2025), including a planner that repairs infeasible plans from a Z3 unsatisfiable core (Hao et al., 2024); multi-agent debate improves factuality and diversity of reasoning (Du et al., 2024; Liang et al., 2024; Chan et al., 2024); and self-refinement can improve outputs (Madaan et al., 2023; Shinn et al., 2023), although self-correction without external feedback often fails (Huang et al., 2024). CADENCE uses each technique in a constrained form: retrieval as case-based reasoning over ADRs (Lewis et al., 2020; Hatalis et al., 2025; Wiratunga et al., 2024), debate restricted to a fixed tactic vocabulary, and critique blended with a solver-derived score so that it is never the only feedback.

Table 1: CADENCE compared with the closest prior work. "Alternatives from" is where candidate decisions come from; "Selection" is how one is chosen; "Feasibility" is whether constraint satisfaction is checked formally.

Approach	Alternatives from	Selection	Feasibility	Precedents
CBAM (Kazman et al., 2001); Al-Naeem et al. (2005)	Stakeholders	Benefit/cost ranking; integer programming	Budget only	No
PerOpteryx (Koziolek et al., 2011)	Architecture model + tactic operators	Evolutionary multi-objective search	Model-based prediction	No
Zimmermann et al. (2009)	Reusable decision model	Architect, rule-checked	Integrity constraints	Decision model
Dhar et al. (2024); Context Matters (Gupta et al., 2026b)	Single LLM	—	No	Retrieved ADRs (Context Matters)
MAAD (Zhang et al., 2025; Li et al., 2025)	Role-pipeline agents	LLM evaluator	No	Retrieved design knowledge
Diaz-Pace et al. (2025)	Exploring design agents	Agent reflection	No	No
CADENCE	Quality-attribute advocate agents	Weighted MaxSAT/SMT, lexicographic	**Solver-checked, with repair**	Retrieved mined ADRs

3. The CADENCE Algorithm

3.1. Problem Formulation and Overview

CADENCE takes as input a natural-language decision context c (the requirements and situational description a real ADR's "Context" section would state) and a set of required quality attributes $Q = \{q_1, \ldots, q_5\}$ drawn from the five ISO/IEC 25010-derived quality attributes fixed for this work: performance, security, maintainability, scalability, and cost/operability. It produces a finalized, provenance-complete ADR: a decision statement, a rationale, a per-quality-attribute utility score, a feasibility verdict with selected tactics and residual gaps, and a full transcript of the precedents consulted, agent positions debated, and repair iterations taken. The four stages — retrieval, deliberation, solver verification with repair, and self-critique finalization — execute in sequence, each consuming the previous stage's output and addressing a distinct failure mode the others do not cover. Figure 1 shows the overall flow, Figure 2 the component architecture behind it, Figure 3 how that architecture maps onto the released code, and Algorithm 1 gives it formally. Appendix A lists the full tactic catalog and Appendix B every prompt, so the method is fully specified in this paper.

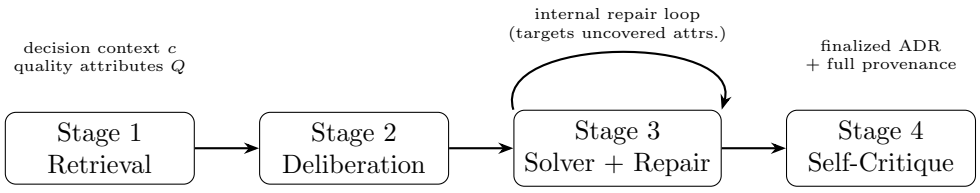


Figure 1: The CADENCE pipeline. Each stage consumes the previous stage's output; Stage 3's repair loop is internal to Stage 3 — it never re-enters Stage 2's multi-agent deliberation — and issues a standalone repair call driven by the solver's uncovered-attribute signal (with the unsatisfiable core reported only as a supplementary diagnostic), iterating to a fixed cap before Stage 4 finalizes the decision.

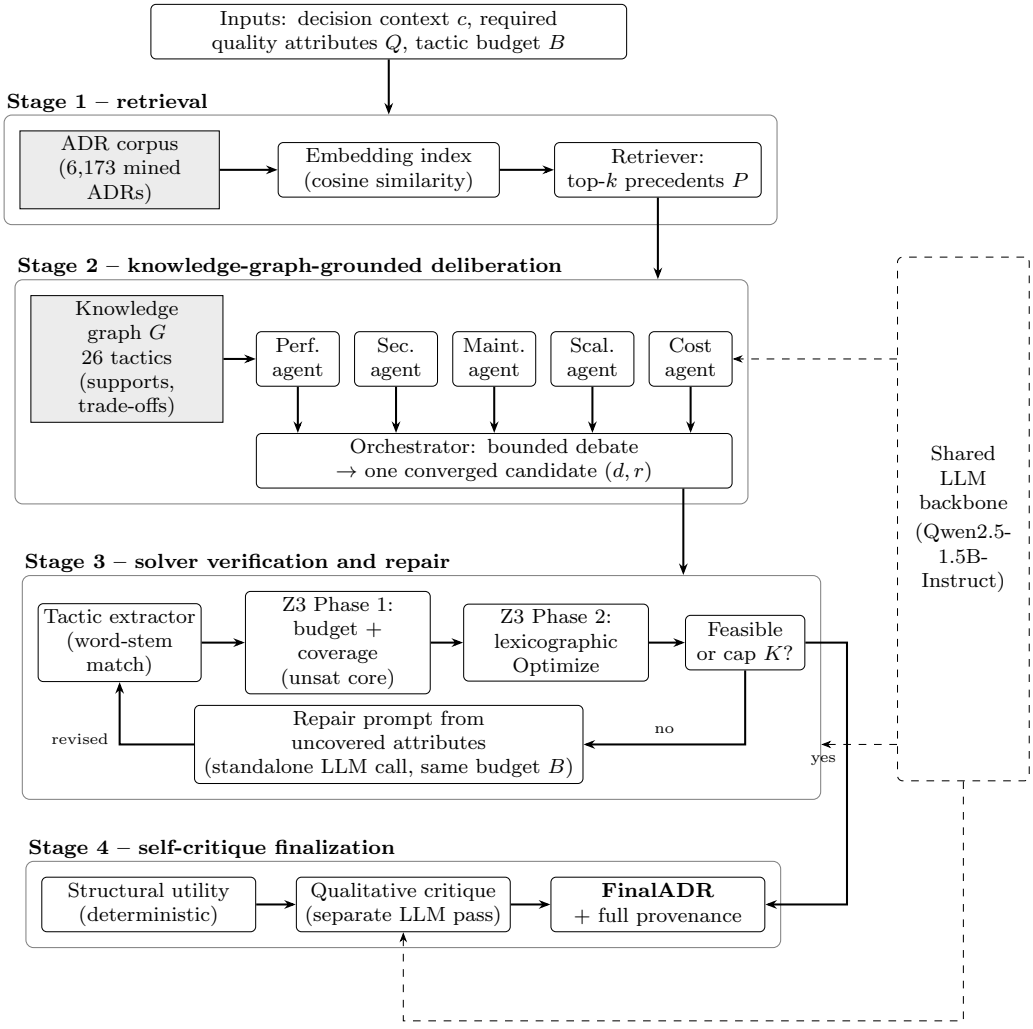


Figure 2: Component architecture of CADENCE. Solid arrows are data flow, dashed arrows are calls to the shared LLM backbone. Stage 1 reads the mined ADR corpus through an embedding index; Stage 2 composes five quality-attribute agents grounded in the tactic knowledge graph G under one orchestrator; Stage 3 turns the candidate into a Z3 instance and loops through a standalone repair call until the instance is feasible or the repair cap K is reached; Stage 4 combines a deterministic structural score with a separate LLM critique into the final ADR with full provenance.

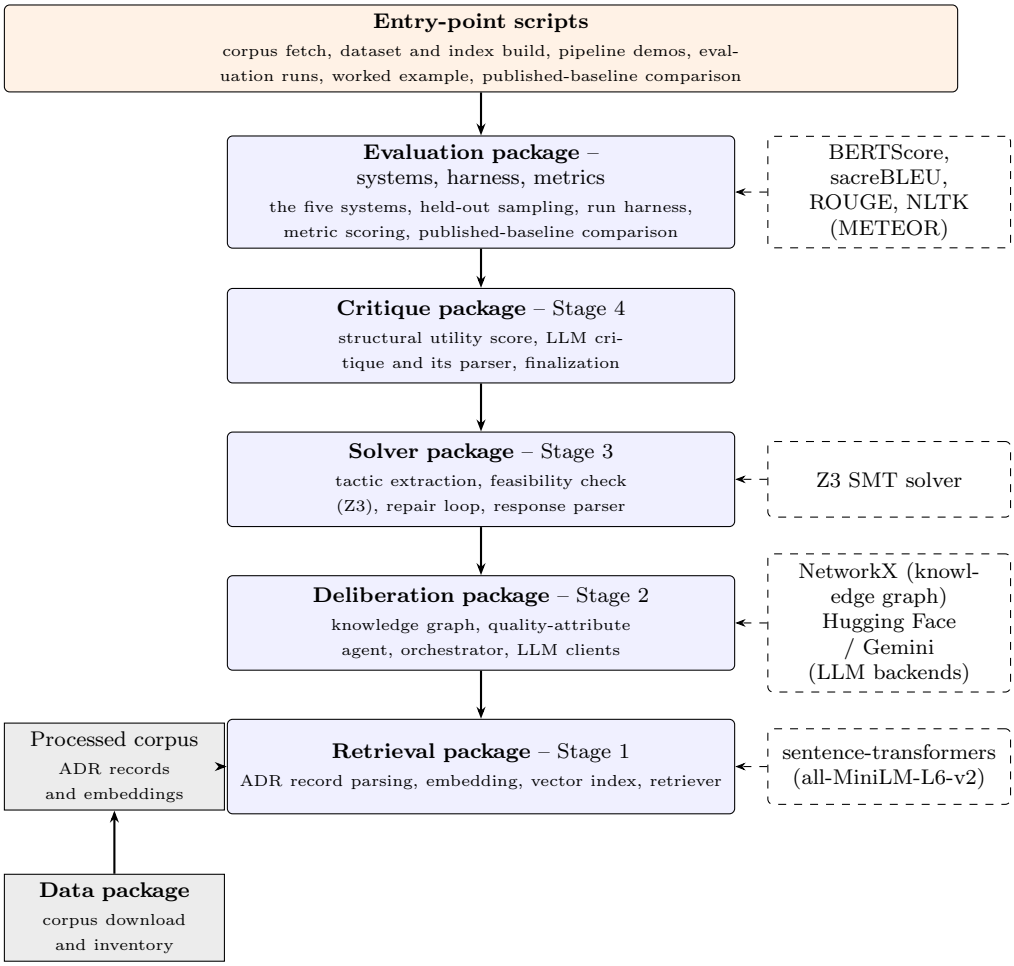


Figure 3: Software architecture of the released implementation, as a stack of Python packages. Solid arrows are import dependencies between adjacent layers (no package imports from a package above it); dashed arrows are third-party libraries. The retrieval, deliberation, solver and critique packages implement Stages 1–4; the evaluation package re-implements the four baselines and computes the metrics; the data package fetches and inventories the ADR corpus that the retrieval package parses.

3.2. Stage 1: Retrieval-Augmented Case-Based Reasoning

Stage 1 embeds the decision context c with a sentence-embedding model and retrieves the k nearest precedent ADRs, by cosine similarity, from a dense vector index built over the full corpus of 6,173 real, mined ADRs (Section 4). Each corpus record is parsed leniently from its source Markdown

Algorithm 1 CADENCE

Require: context c, quality attributes Q, tactic budget B, tactic catalog with graph G, max deliberation rounds R, max repair iterations K
Ensure: finalized decision with provenance

$P \leftarrow \textsc{Retrieve}(c)$ {Stage 1: top-k precedents}
$(d, r) \leftarrow \textsc{Deliberate}(c, P, Q, G, R)$ {Stage 2: bounded multi-agent debate}
$i \leftarrow 0$; $best \leftarrow \perp$
loop
  $res \leftarrow \textsc{CheckFeasibility}(d, Q, B, G)$ {Stage 3, Phase 1+2}
  if $best = \perp$ **or** $|res.covered| > |best.res.covered|$ **then**
    $best \leftarrow (res, d, r)$ {track highest-coverage attempt seen so far}
  end if
  if $res.feasible$ **or** $i = K$ **then**
    break {degrade gracefully with a caveat if infeasible}
  end if
  $(d, r) \leftarrow \textsc{Repair}(d, res.uncovered, B)$ {standalone LLM repair call, targets uncovered attributes}
  $i \leftarrow i + 1$
end loop
$(res, d, r) \leftarrow best$ {use the best attempt seen, not necessarily the last}
$u \leftarrow \textsc{StructuralUtility}(res, G)$ {Stage 4, deterministic}
$q \leftarrow \textsc{QualitativeCritique}(d, r, Q)$ {Stage 4, separate LLM pass}
return $\textsc{Finalize}(d, r, u, q, res, P)$

file — real-world ADR filenames and internal structure are inconsistent across repositories, so the parser extracts only a title and the full raw body text rather than assuming a fixed section schema. Retrieval returns the top-k precedent records, whose titles are passed into Stage 2 as grounding context; when used inside a held-out evaluation, the query record's own ground-truth ADR is excluded from its own retrieval results to prevent leakage.

3.3. Stage 2: Knowledge-Graph-Grounded Multi-Agent Deliberation

Stage 2 instantiates one agent per required quality attribute in Q. Each agent is grounded in a knowledge graph G built from a hand-curated catalog of 26 architectural tactics spanning the five quality attributes (5 or 6 per attribute; Table 2 lists the full catalog), where each tactic node has a di-

rected *supports* edge to the quality attribute it primarily serves and directed *trade-off* edges, annotated with a natural-language explanation, to every other attribute it is known to burden — for instance, *caching* supports *performance* but trades off against *maintainability* via cache-invalidation complexity, and *authentication* supports *security* but trades off against both *performance* (verification latency) and *cost_operability* (identity-infrastructure upkeep), as Figure 4 depicts. Given the decision context and Stage 1's retrieved precedents, each agent proposes a candidate decision serving its own attribute using G's *supports* edges, and may critique candidates proposed by other agents; G's *trade-off* edges are consumed downstream by Stage 3's feasibility optimization and Stage 4's utility score, rather than injected into Stage 2's prompts. Deliberation proceeds for a bounded number of rounds (R), after which an orchestrator synthesizes the debate into one converged candidate decision and rationale.

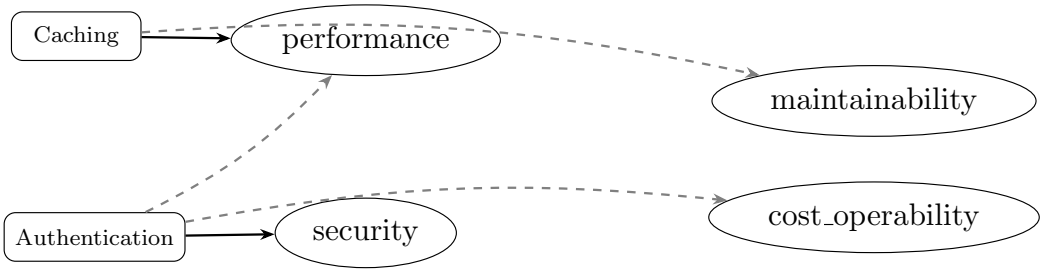


Figure 4: Excerpt of knowledge graph G: solid arrows are *supports* edges, dashed gray arrows are *trade-off* edges. Only 2 of the catalog's 26 tactics and 4 of its 5 quality attributes are shown for legibility.

Table 2: The 26-Tactic Catalog Grounding Stage 2's Knowledge Graph G. The number in parentheses is the count of *trade-off* edges the tactic carries to other quality attributes (0–2 in this catalog); each tactic's *supports* edge targets the quality attribute in its row.

Quality attribute	Tactics (trade-off edges)
Performance	Caching (1); connection pooling (1); asynchronous processing via message queues (2); data denormalization (1); read-through/write-behind access (1)
Security	Authentication (2); fine-grained authorization (1); encryption in transit/at rest (2); rate limiting/throttling (1); least-privilege service boundaries (1); input validation at trust boundaries (1)
Maintainability	Modular decomposition by capability (2); versioned API contracts (1); automated regression test suite (1); centralized structured logging (2); dependency injection/IoC (1)
Scalability	Horizontal scaling with stateless instances (2); database sharding (2); event-driven architecture (2); auto-scaling infrastructure (1); read replicas (1)
Cost/operability	Managed cloud services (2); infrastructure as code (0); multi-tenancy (2); serverless/FaaS deployment (2); circuit breaker for dependencies (1)

Appendix A gives each tactic's effect and every trade-off edge with its rationale.

Synthesis is the stage's most format-fragile step: it asks the underlying LLM to emit a two-field response (a one- or two-sentence candidate decision, then a one-paragraph rationale) that Stage 3 can parse deterministically. In practice, real small-model output deviates from any single expected format in ways a fixed parser cannot fully anticipate in advance (Section 3.6); the orchestrator therefore retries the same synthesis prompt up to three times on a parse failure before surfacing an error, since generation is stochastic and a differently-sampled retry frequently self-corrects a formatting slip that a hand-written regex extension would not have anticipated.

3.4. Stage 3: Constraint-Solver-Verified Feasibility with Repair Loop

Stage 2's converged candidate is free text: a human-readable decision and rationale, not a structured commitment a solver can reason about directly. Stage 3 first extracts which of the 26 catalog tactics the candidate text actually mentions, by word-stem overlap: a tactic counts as mentioned if at least 60% of the distinct significant words of its name (longer than three letters, excluding stopwords, truncated to four-letter stems) occur among the stems of the candidate text, giving Boolean decision variables $\{t_1, \ldots, t_m\}$, one per

mentioned tactic. It then checks feasibility in two phases against the Z3 SMT solver.

Phase 1 (strict feasibility). A budget constraint $\sum_i t_i \leq B$ (a caller-specified tactic budget B) and, for every required quality attribute $q \in Q$, a coverage constraint $\bigvee_{i\,:\,\text{supports}(t_i)=q} t_i$ (at least one selected tactic must support q) are asserted as individually tracked constraints, so that if the instance is unsatisfiable, Z3 returns an *unsatisfiable core*: the constraints jointly responsible for infeasibility. This core is a useful, but not necessarily complete, diagnostic — Z3's core-extraction semantics can name only one of several symmetric zero-coverage attributes as blocking, omitting others equally unsatisfied — so Stage 3 does not treat it as authoritative.

Phase 2 (best-effort optimized selection). A separate optimization instance re-asserts the budget constraint and searches for the tactic selection that jointly (a) maximizes how many required quality attributes are covered and (b) minimizes the caller-weighted cost of the trade-offs incurred, always returning a concrete selection even when Phase 1 found the instance strictly infeasible. These two objectives are declared to Z3 as separate groups of soft constraints under its lexicographic priority mode, rather than one weighted-sum objective — a distinction we verified matters empirically. An earlier weighted-sum formulation used a single large constant to bias coverage above trade-off cost, but because trade-off weights are caller-supplied with no upper bound, a selection accumulating enough trade-off weight could mathematically outvote a required attribute's coverage regardless of the bias constant's size (a concrete repro during development: four trade-offs at weight 300 each outweighed a coverage bias of 1000). Lexicographic priority makes coverage strictly dominate trade-off avoidance independent of weight magnitude — the only encoding safe for caller-supplied weights whose scale cannot be bounded in advance. The resulting split of the required attributes into a covered set and an uncovered set from this phase, not Phase 1's unsat core, is the sound and complete signal of what still needs fixing.

If Phase 1 finds the instance infeasible, Stage 3 constructs a repair prompt naming, for every uncovered attribute, the concrete tactics that would cover it, and asks the LLM to revise the candidate within the same budget. This is re-checked against the solver, iterating up to K times and tracking the best (highest-coverage) attempt seen. If the budget is never satisfiable within the cap, Stage 3 returns the best attempt found with an explicit caveat rather than silently accepting an infeasible decision or discarding an otherwise-

reasonable candidate; a transient LLM or parse failure during any repair attempt triggers the next iteration, not an abort.

3.5. Stage 4: Self-Critique Finalization

Stage 4 combines two independently-computed signals into each quality attribute's final utility score. The *structural* component is deterministic: for attribute q, it is 1.0 if q is among the solver-verified decision's covered attributes, minus a fixed penalty of 0.2 for every incoming trade-off edge from a selected tactic onto q in G, floored at 0.0; it depends only on Stage 3's already-verified output and involves no further LLM call. The *qualitative* component is a single LLM pass scoring every required attribute on a 0–10 scale and flagging any residual weakness, given the full decision, rationale, and attribute list in one call; parsing uses a three-tier matcher — a strict same-line pattern, a tolerant same-line fallback, and a heading-section fallback for responses that state an attribute as its own heading with the field on a separate, unlabeled line (Section 3.6) — so a loose regex cannot lock onto a prose preamble instead of the real scored line, and the call is retried up to three times on a parse failure as Stage 2's synthesis call is. The two components combine as $\text{combined}_q = 0.5 \cdot \text{structural}_q \cdot 10 + 0.5 \cdot \text{qualitative}_q$, and overall decision utility is the mean combined score across all required attributes. The finalized ADR record carries this scoring together with full provenance — retrieved precedents, the complete per-agent deliberation transcript, which tactics were selected and by which repair iteration, and any solver caveat — so the decision is auditable end-to-end.

3.6. Implementation Robustness: Parsing Real, Stochastic LLM Output

A structured-output prompt's hand-written happy-path parser reliably breaks against a real, deployed small language model, and our development process surfaced this repeatedly and concretely, not as a theoretical concern. Running the full pipeline end-to-end against Qwen2.5-1.5B-Instruct, the reproducible local backbone used throughout this work, produced six distinct real formatting deviations across Stages 2–4, summarized in Table 3.

Table 3: Six Real LLM Output-Format Deviations Encountered

#	Deviation	Resolved by
1	Extra descriptor word ("Candidate Decision:" for "Candidate:")	Tolerant same-line parser
2	Non-literal "no weakness" ("None identified" for literal "none")	Tolerant same-line parser
3	Fraction-style score ("8/10"), this model's *default* 0–10 answer	Tolerant same-line parser
4	Prose preamble a loose regex could lock onto instead of the real line	Tolerant same-line parser
5	Genuine word substitution ("Candidacy:" for "Candidate:"), unanticipatable by regex tolerance	Bounded retry (2–3 attempts)
6	Attribute-as-heading structure: attribute on its own heading line, field unlabeled on a later line	Third parser tier (heading-section fallback)

The first four are addressable by a sufficiently tolerant same-line parser. The fifth is not, and its discovery motivated this work's general strategy of pairing every structured-output prompt with a bounded retry (two to three attempts) of the identical prompt on a parse failure, rather than special-casing wording variants indefinitely; because generation is stochastic, a differently-sampled retry frequently self-corrects a slip the parser did not anticipate, and Stages 2, 3, and 4 all apply this pattern. The sixth, discovered later during the scaled evaluation run (e.g. `**Performance**` followed, several lines later, by `**Score:** 9/10` with no attribute name on that line), shows retry's limit: it exhausted all three attempts because it was not a stochastic one-off but a genuinely different *structure* no same-line pattern, however tolerant, could match — fixing it required a third parser tier (Section 3.5) that locates an attribute's heading and searches only the bounded section up to the next heading for an unlabeled field. We report this as a methodological finding in its own right: any system asking an LLM for structured intermediate output a downstream deterministic component must consume should validate its parser against real, repeated model output before trusting it unattended, treat bounded retry as the general-purpose fix for wording variation, and expect that a genuinely new structural format will eventually require a genuinely new parser tier — discoverable only by actually running the real pipeline.

3.7. Worked Example

To make the four stages concrete, this section walks through one real, complete run of the pipeline reported as produced by the implementation — illustrative, not an evaluation data point; this paper's quantitative claims come from Section 4's held-out runs, not this example.

Given a context describing a 10x read-traffic surge on an order-processing service, with requirements to keep p99 latency low, stay operable by a small team, and avoid new security risk, **Stage 1** retrieves three real precedents on message brokering, server architecture, and JWT inter-service authentication. **Stage 2** runs two rounds across the five agents: the *performance* agent proposes a message queue to decouple reads from the application server; the *security* agent independently argues for JWT authentication, encryption, and rate limiting; the *cost_operability* agent's round-2 critique flags a real cross-attribute tension, that the message broker's added network overhead "could affect real-time interactions." The orchestrator synthesizes these into one converged candidate, from which Stage 3 extracts a tactic selection.

Stage 3 extracts three catalog tactics — *asynchronous processing via message queues*, *event-driven architecture*, and *serverless/FaaS deployment* — covering *performance*, *scalability*, and *cost_operability*, but not *maintainability* or *security*. This example runs at $B = 4$, which, as Section 4 establishes, cannot cover all five attributes under a catalog where each tactic supports exactly one — unlike the scaled run's $B = 5$ condition, it is not a test of achievable-budget feasibility. It nonetheless illustrates the repair loop's mechanics: one tactic slot remained within budget, but the candidate's text never named a fourth, security- or maintainability-covering tactic for the extractor to find. Both repair attempts fail to close the gap, and Stage 3 returns the best-effort selection with an explicit caveat naming the covered and unaddressed attributes.

Stage 4 finalizes the decision — a microservices architecture using a message queue, an event-sourcing database, and serverless functions — and scores it per attribute (Table 4; Figure 5 draws the resulting architecture), flagging a concrete weakness for each (e.g., for *security*: serverless functions carry function-timeout limits and reduced control over the execution environment). The two uncovered attributes' structural components are correctly 0.0, dragging their combined scores down even where the qualitative pass alone would have scored the prose favorably — exactly the blending Section 3.5 argues for.

Table 4: Worked Example (Illustrative, $B = 4$, Not an Evaluation Data Point): Per-Attribute Utility Scores (Overall: 6.6/10)

Quality Attribute	Structural	Qualitative	Combined
Performance	0.8	8.0	8.0
Security	0.0	9.0	4.5
Maintainability	0.0	7.0	3.5
Scalability	1.0	8.0	9.0
Cost/Operability	1.0	6.0	8.0

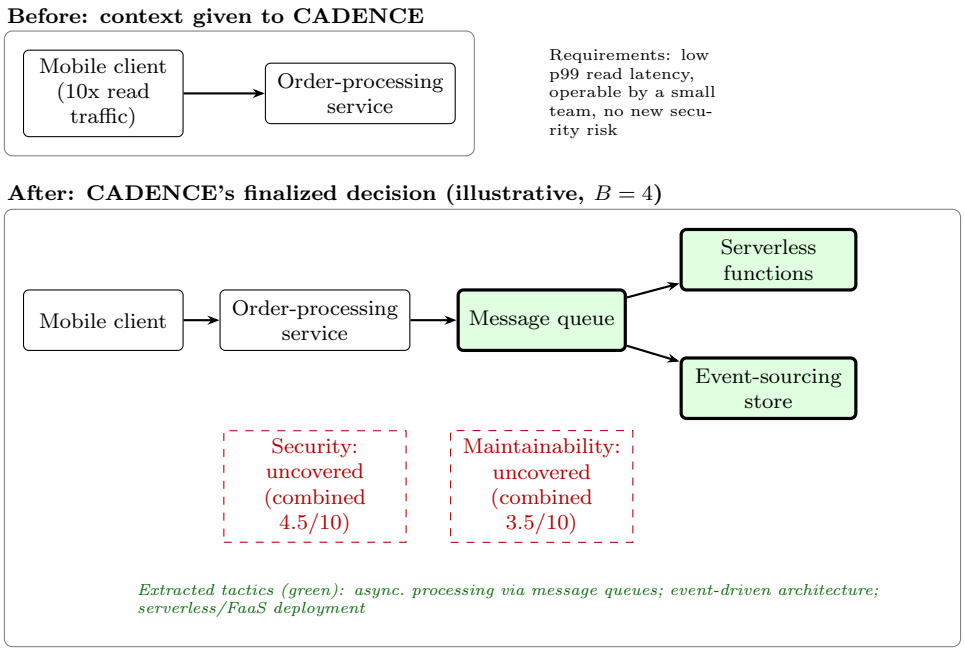


Figure 5: The worked example as an architecture. Top: the context given to CADENCE. Bottom: the components named in the finalized decision (green: tactics the extractor found) and the two quality attributes the decision leaves uncovered (red, with their combined utility scores from Table 4). The topology is schematic: the run reports which components were selected, not how they are wired.

4. Evaluation

4.1. Study Design

Corpus and task. We evaluate against the "Context Matters" replication package (Gupta et al., 2026a) (Zenodo, DOI: 10.5281/zenodo.18370195), which is derived from the same mining study that provides Stage 1's retrieval corpus (Buchgeher et al., 2023). The package catalogs 954 repository folders, each with an extraction-confidence status: 750 *Verified*, 96 *Doubt (name sequence)*, 40 *Repo Inaccessible*, 33 *Doubt (no repo dir)*, 30 *Doubt (missing file)* and 5 *Doubt (file contents)*. After parsing, the 882 folders with at least one accessible Markdown file yield 6,173 ADR files, with a median of 4, a mean of 7.00 and a maximum of 129 per repository. Held-out items are drawn only from *Verified* folders, so that the quality of the ground truth is not itself a confound. We follow the corpus's own task formulation, "ADR generation from titles": each query is a record's title, and the record's full body is the ground truth. A query record is always excluded from its own retrieval results.

Sample. We draw $N = 30$ held-out records (seed 44) uniformly from the *Verified* records with at least 300 characters of body text. The six records used in two earlier $N = 3$ runs during development (seeds 42 and 43) are excluded, so the sample is disjoint from every item we inspected while building the system. All 30 items completed end to end, with no parse or critique failures.

Configuration. The LLM backbone for every system and every stage is Qwen2.5-1.5B-Instruct, run locally through Hugging Face Transformers on one NVIDIA T4 GPU. Decoding is sampled at temperature 0.7 with at most 400 new tokens per call. Retrieval returns $k = 3$ precedents. Deliberation runs $R = 2$ rounds (proposals, then one critique round) with the five quality-attribute agents of Section 3. The repair loop has $K = 2$ attempts. All five attributes are required, with equal weights. We test two tactic budgets. $B = 5$ is the smallest feasible budget: each tactic supports exactly one attribute, so five attributes need at least five tactics. $B = 7$ adds two tactics of slack, which tests whether the budget, rather than the candidate's content, limits feasibility. One item took about 5 minutes end to end, and the run took 2.5 hours.

Paired design. For each item, Stages 1 and 2 run once, and every downstream system consumes that same output. Multi-Agent is the deliberation's candidate. Stage 3 is then run on that candidate once per budget, giving Cadence-NC, and Stage 4 runs on the resulting verified decision, giving Cadence-Full. Adjacent systems therefore differ in exactly one stage on identical input, and the two budget conditions share one deliberation. The two earlier development runs did not have this property, because each system re-ran Stages 1–3 independently; Section 4.8 explains why this matters.

4.2. Systems Compared

The systems form an ablation ladder in which each step adds one stage:

- Zero-Shot: a single LLM call given only the decision context, the baseline style of Dhar et al. (2024).
- Retrieval-Only: the context plus the titles of Stage 1's top-k precedents, the retrieval-augmented style of Gupta et al. (2026b).
- Multi-Agent: Stages 1–2. This is the deliberation's converged candidate and rationale, comparable to multi-agent ADR generation such as MAAD (Zhang et al., 2025) but with quality-attribute advocates instead of functional roles.
- Cadence-NC: Stages 1–3. The candidate after solver verification and, where it is infeasible, repair.
- Cadence-Full: Stages 1–4. Stage 4 scores the verified decision; it does not rewrite it (Section 3.5). The decision and rationale text of Cadence-Full is therefore identical to that of Cadence-NC. What Stage 4 adds is the per-attribute utility scores and the residual weaknesses. To let the text metrics see that addition, we also score Cadence-Full+Cons., in which the residual weaknesses are appended as a "Consequences" section. This is the section in which an ADR conventionally records the negative effects of a decision (Tyree and Akerman, 2005).

4.3. Measures and Analysis

Agreement with the recorded decision. We report the four metrics used in the LLM-for-ADR studies we compare against (Dhar et al., 2024; Gupta et al., 2026b): BERTScore F1 (roberta-large, no baseline rescaling), sentence-level BLEU (sacreBLEU, 0–100), ROUGE-1 F1 (with stemming) and METEOR. Each output is compared with the human-authored body of the held-out ADR. These metrics measure agreement with one recorded decision, not

decision quality. Their limits for software-engineering text are well documented (Roy et al., 2021; Evtikhiev et al., 2023; Haque et al., 2022), and RQ4 examines them directly.

Feasibility. For Stage 3 we report the constraint-satisfaction rate (the share of items for which the decision covers all required attributes within B) and the number of attributes covered. We also report the coverage of the deliberation's candidate *before* repair, by applying Stage 3's deterministic tactic extraction to it. This separates what deliberation contributes from what repair contributes.

Statistics. Every comparison is paired by item. For each adjacent pair on the ladder we report the mean paired difference with a 95% percentile bootstrap interval (10,000 resamples), the count of items on which each system wins, and a two-sided Wilcoxon signed-rank test. Within each metric, we adjust the p-values for the eleven comparisons with Holm's method. A difference is called significant when the Holm-adjusted $p < 0.05$.

4.4. RQ1: What Does Each Stage Contribute?

Table 5: Mean agreement with the held-out human-authored ADRs ($N = 30$, paired), with 95% bootstrap intervals, and median output length in words (reference ADRs: median 163, mean 289).

System	BERTScore F1	BLEU	ROUGE-1 F1	METEOR	Words
Zero-Shot	0.813 [0.809, 0.818]	1.90 [1.41, 2.47]	0.255 [0.232, 0.275]	0.223 [0.202, 0.243]	301
Retrieval-Only	0.817 [0.813, 0.821]	1.90 [1.41, 2.48]	0.265 [0.246, 0.283]	0.227 [0.206, 0.246]	300
Multi-Agent	0.804 [0.799, 0.810]	0.56 [0.34, 0.81]	0.144 [0.122, 0.168]	0.084 [0.068, 0.103]	90
Cadence-NC, $B = 5$	0.799 [0.794, 0.803]	0.58 [0.40, 0.79]	0.148 [0.132, 0.163]	0.093 [0.077, 0.109]	120
Cadence-Full, $B = 5$	0.799 [0.794, 0.803]	0.58 [0.40, 0.79]	0.148 [0.132, 0.163]	0.093 [0.077, 0.109]	120
Cadence-Full+Cons., $B = 5$	0.802 [0.798, 0.806]	0.66 [0.50, 0.84]	0.178 [0.161, 0.193]	0.134 [0.120, 0.148]	233
Cadence-NC, $B = 7$	0.802 [0.797, 0.807]	0.58 [0.37, 0.83]	0.147 [0.128, 0.168]	0.089 [0.074, 0.105]	107
Cadence-Full, $B = 7$	0.802 [0.797, 0.808]	0.58 [0.37, 0.83]	0.147 [0.128, 0.168]	0.089 [0.074, 0.105]	107
Cadence-Full+Cons., $B = 7$	0.805 [0.800, 0.810]	0.66 [0.48, 0.85]	0.175 [0.157, 0.191]	0.124 [0.110, 0.139]	209

Table 6: Paired differences along the ablation ladder ($N = 30$): mean difference (later minus earlier system), with wins/losses of the later system in parentheses; ties are omitted. *: Holm-adjusted Wilcoxon $p < 0.05$ within the metric.

Step (stage added)	BERTScore F1	BLEU	ROUGE-1 F1	METEOR
Zero-Shot → Retrieval-Only (retrieval)	+0.004* (22/8)	+0.01 (15/15)	+0.010 (21/9)	+0.004 (15/15)
Retrieval-Only → Multi-Agent (deliberation)	−0.013* (3/27)	−1.35* (4/26)	−0.120* (3/27)	−0.142* (0/30)
Multi-Agent → Cadence-NC, $B = 5$ (solver, repair)	−0.006* (0/17)	+0.02 (14/3)	+0.003 (12/5)	+0.008 (14/3)
Multi-Agent → Cadence-NC, $B = 7$ (solver, repair)	−0.002 (5/9)	+0.02 (9/4)	+0.002 (9/4)	+0.005 (10/3)
Cadence-NC → Cadence-Full, both B (critique)	0 (0/0)	0 (0/0)	0 (0/0)	0 (0/0)
Cadence-NC → Cadence-Full+Cons., $B = 5$	+0.003* (22/8)	+0.07 (18/12)	+0.031* (21/9)	+0.041* (27/3)
Cadence-NC → Cadence-Full+Cons., $B = 7$	+0.003* (26/4)	+0.08 (18/12)	+0.028* (22/8)	+0.035* (29/1)
Zero-Shot → Cadence-Full, $B = 5$ (whole pipeline)	−0.015* (3/27)	−1.31* (7/23)	−0.107* (1/29)	−0.130* (0/30)

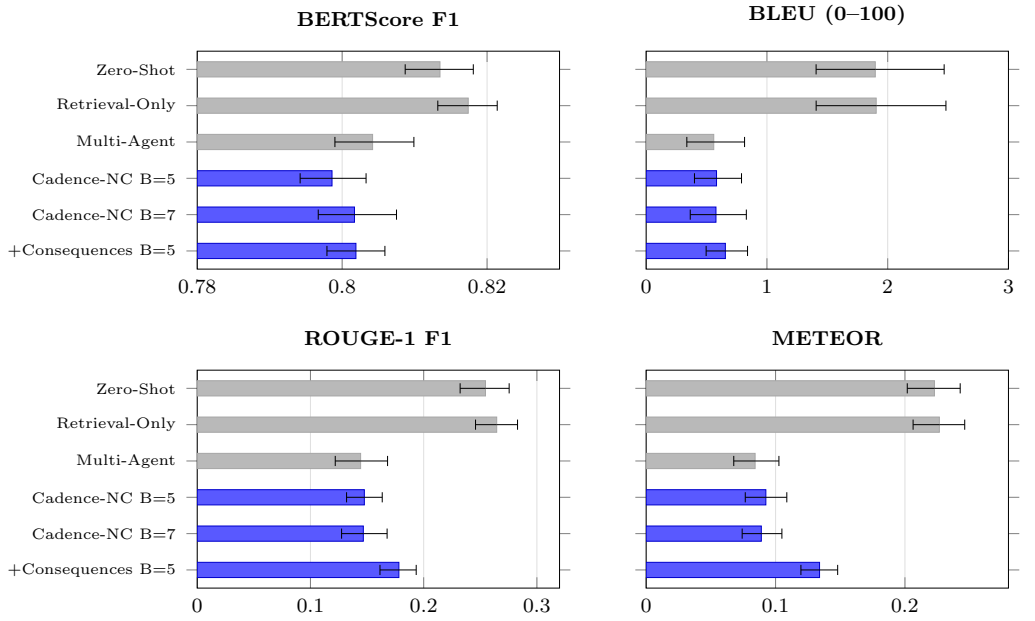


Figure 6: Mean agreement with the held-out ADRs and 95% bootstrap intervals ($N = 30$). Grey: single-pass and deliberation-only systems; blue: systems that include Stage 3. Cadence-Full is omitted because its text equals Cadence-NC's. The BERTScore axis starts at 0.78.

Tables 5 and 6 and Figure 6 report the results.

Retrieval gives a small but consistent gain in BERTScore (+0.004, 22 of 30 items, significant). It does not change the surface metrics significantly.

Deliberation is the largest effect in the study, and it is negative on every metric. Against Retrieval-Only, Multi-Agent loses on BERTScore in 27 of 30 items and on METEOR in all 30. Its output is also much shorter (median 90 words against 300). This is by design: the synthesis prompt asks for a one-line decision and a short rationale that Stage 3 can parse. RQ4 tests how much of the loss is due to length alone.

Solver verification and repair leave the decision text unchanged on the 13 items (at $B = 5$) where repair did not improve coverage, because the loop then returns the original candidate. On the 17 changed items, repaired text is slightly closer to the reference on the surface metrics (14 of 17 items on BLEU and METEOR), but no such difference is significant. BERTScore drops slightly but significantly at $B = 5$ (-0.006). The solver stage is thus roughly neutral for agreement with the recorded decision. Its value has to be judged on feasibility (RQ2).

Critique does not change the text at all, so its difference on every metric is exactly zero. Rendering Stage 4's residual weaknesses as a Consequences section improves BERTScore, ROUGE-1 and METEOR significantly at both budgets; METEOR improves on 27 and 29 of 30 items. The section roughly doubles the output length, however, so this is partly a length effect (RQ4). It does indicate that what the critique identifies overlaps with what architects actually wrote down.

Taken as a whole, the full pipeline agrees less with the recorded ADRs than one zero-shot call does, on all four metrics.

4.5. RQ2: Does Solver Verification Make Decisions Feasible?

Table 7: Solver outcomes ($N = 30$). Coverage is the number of required quality attributes (of five) covered within the budget.

	Before repair	After repair, $B = 5$	After repair, $B = 7$
Feasible items	0/30	1/30	0/30
Mean attributes covered	0.57	1.63	1.23
Items with zero coverage	14	6	8
Items whose coverage repair improved	—	17	13
Mean repair attempts used	—	2.00	2.00

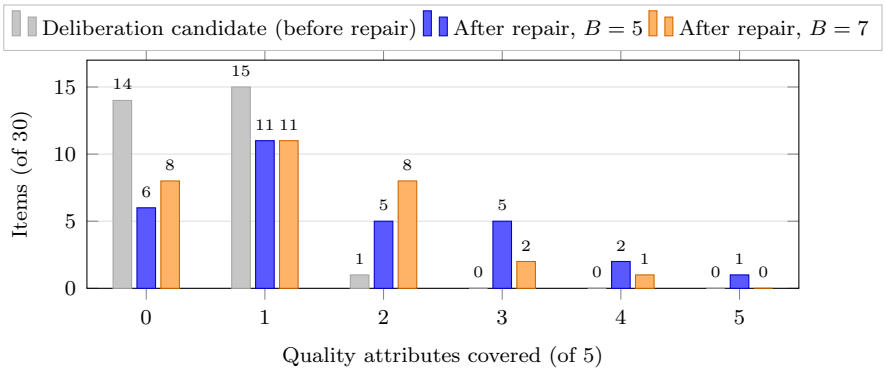


Figure 7: Number of required quality attributes (of five) covered by the decision's tactic commitments, per item ($N = 30$): the deliberation's candidate before repair, and the best candidate after up to $K = 2$ repairs at each budget. Feasibility requires all five.

Table 7 and Figure 7 report the solver's view of the same 30 decisions.

Deliberated decisions rarely commit to catalog tactics, and when they do, the tactics cluster on one attribute. Before repair, 14 of 30 candidates name no tactic from the catalog, and 15 cover a single attribute. The mean is 0.57 of five. This holds even though every agent's prompt lists the tactics for its attribute (Appendix B.1). The converged decision often keeps only the decision's subject; one candidate reads in full "Akka Persistence". When a candidate does name several tactics, they tend to come from one group. "Use asynchronous processing via message queues, caching, connection pooling, and data denormalization" names four tactics, all of which support performance. The advocates for the other four attributes argued during the rounds, but their concerns did not survive into the synthesized decision.

Repair helps, but not enough. Repair raises mean coverage to 1.63 attributes at $B = 5$ and improves 17 of 30 items. It reaches full coverage once: on one item, the second repair produced a decision naming five tactics, one per attribute. Every other item exhausted both repair attempts.

Slack does not help. At $B = 7$, no item is feasible, and coverage is lower than at $B = 5$ (1.23; 12 items are better at $B = 5$, 7 at $B = 7$). The candidates and solver encoding are identical across budgets, so the difference comes entirely from the stochastic repair calls. The number of tactics is not what binds. Repaired decision statements often name more tactics than either budget allows, up to 18 in one case. What binds is *diversity*: the named tactics support too few distinct attributes. A larger budget cannot

fix a decision that is silent on security.

This is the answer to RQ2 for this backbone: formal verification works as a *detector*. On all 30 items it correctly reports that the decision leaves required attributes unaddressed, and it names which ones. Bounded repair through the same small model does not turn that detection into feasible decisions. Section 5 discusses what this implies for the design.

4.6. RQ3: Comparison with Published Frontier-Model Generations

The comparisons above use our own baseline implementations and one small backbone. For an external reference point, we looked up our 30 held-out items in the "Context Matters" replication package's archive of generated ADRs (Gupta et al., 2026a), matching each item by repository and title. The archive holds generations from four models: Gemini-2.5-Pro (Comanici et al., 2025) (proprietary, size undisclosed), GLM-4.6 (Zeng et al., 2025) (about 357B parameters, mixture-of-experts), Qwen3-235B (Yang et al., 2025) (235B total, 22B active, mixture-of-experts) and Gemma3-4B (Gemma Team et al., 2025) (4B, dense). Each model was run under a title-only strategy and a retrieval-augmented strategy ($k = 3$), against the same ground truth we use. The title-only generations cover 27 of our 30 items. The retrieval-augmented generations cover 15, because the package did not run that strategy for every record.

We do not use the package's precomputed scores. Its evaluation script computes BLEU and METEOR with prediction and reference swapped, relative to standard usage and to our code, and computes ROUGE-1 without stemming. We therefore recompute all four metrics from the package's raw generation and ground-truth texts with exactly our own metric code. Every number in Table 8 is computed the same way, and each comparison uses only the items both sides cover. (GLM-4.6 has no technical report of its own; we cite GLM-4.5, of which it is an incremental update.)

Table 8: Published frontier-model generations from the "Context Matters" package and this work's systems on the same held-out items. All scores were recomputed with this paper's metric code from raw text. Each block uses only the items that the published strategy covers.

System	Size	BERTScore F1	BLEU	ROUGE-1 F1	METEOR
Title-only strategy, 27 items					
Gemini-2.5-Pro	undisclosed	0.826	2.82	0.235	0.258
GLM-4.6	∼357B MoE	0.825	3.55	0.253	0.267
Qwen3-235B	235B (22B active)	0.827	4.50	0.258	0.267
Gemma3-4B	4B	0.819	2.36	0.239	0.240
ZERO-SHOT (this work)	1.5B	0.814	1.85	0.255	0.228
CADENCE-NC, $B = 5$ (this work)	1.5B	0.801	0.63	0.151	0.097
CADENCE-FULL+Cons., $B = 5$ (this work)	1.5B	0.803	0.70	0.179	0.139
Retrieval-augmented strategy ($k = 3$), 15 items					
Gemini-2.5-Pro	undisclosed	0.852	10.61	0.387	0.336
GLM-4.6	∼357B MoE	0.851	10.46	0.379	0.325
Qwen3-235B	235B (22B active)	0.848	9.12	0.352	0.315
Gemma3-4B	4B	0.834	5.65	0.307	0.269
RETRIEVAL-ONLY (this work)	1.5B	0.822	1.76	0.286	0.218
CADENCE-NC, $B = 5$ (this work)	1.5B	0.801	0.43	0.141	0.086
CADENCE-FULL+Cons., $B = 5$ (this work)	1.5B	0.805	0.57	0.187	0.134

Two observations follow from Table 8. First, our single-pass baselines are not artificially weak. Under the title-only strategy, ZERO-SHOT with a 1.5B backbone matches the published models on ROUGE-1 (0.255 against 0.235–0.258). It trails the frontier models on BERTScore by about 0.011 (winning on 5–6 of 27 items) and is close to Gemma3-4B, a model about 2.7 times its size. Second, CADENCE's gap to the published systems is the same gap it has to our own baselines, only wider. On BERTScore, CADENCE-NC trails every published model on 23 to 26 of 27 title-only items, and on all or all but one of the 15 retrieval-augmented items. Retrieval also helps the published models much more than our title-only precedents help RETRIEVAL-ONLY. On the 15 items both strategies cover, their BERTScore rises by 0.013 to 0.026 from title-only to retrieval-augmented generation; ours rises by 0.003. A strong retrieval baseline is therefore the bar any LLM-for-ADR approach has to clear on reference agreement. On this measure CADENCE does not clear it with a small backbone, for the reasons analyzed in RQ4.

4.7. RQ4: How Well Do Reference-Based Metrics Capture the Difference?

Table 9: Length-controlled comparison ($N = 30$). Each baseline's output is truncated, per item, to the word count of CADENCE-NC ($B = 5$) on that item. Differences are CADENCE-NC minus the truncated baseline; wins/losses of CADENCE-NC in parentheses; *: Holm-adjusted $p < 0.05$.

	BERTScore F1	BLEU	ROUGE-1 F1	METEOR
ZERO-SHOT, full length	0.813	1.90	0.255	0.223
ZERO-SHOT, truncated	0.812	1.80	0.234	0.177
RETRIEVAL-ONLY, truncated	0.814	1.81	0.246	0.178
CADENCE-NC, $B = 5$	0.799	0.58	0.148	0.093
CADENCE-NC vs. ZERO-SHOT, truncated	-0.013^* (2/28)	-1.22^* (1/29)	-0.087^* (0/30)	-0.085^* (0/30)
CADENCE-NC vs. RETRIEVAL-ONLY, truncated	-0.015^* (3/27)	-1.23^* (0/30)	-0.098^* (0/30)	-0.086^* (0/30)

CADENCE's outputs are 2.5–3 times shorter than the baselines' (Table 5). Length does track the surface metrics: across all systems and items, the ratio of output length to reference length correlates with ROUGE-1 (Spearman $\rho = 0.26$, $p < 10^{-4}$, 270 system–item pairs). To isolate length, Table 9 truncates each baseline's output to CADENCE-NC's length on the same item.

Truncation closes only part of the gap. Against ZERO-SHOT, length accounts for about a fifth of the ROUGE-1 gap (from 0.107 to 0.087) and a third of the METEOR gap (from 0.130 to 0.085). At equal length, the baselines still win on every metric and on nearly every item, BERTScore included. In the earlier, $N = 3$ version of this work, we read the gap between surface metrics and BERTScore as an artifact of output length. That reading does not survive the length control.

The remaining gap reflects what the text says. An ADR as architects write it has a context, a decision, and consequences, in project-specific vocabulary. The single-pass baselines imitate that form. CADENCE's output is a decision statement plus a rationale phrased in quality-attribute and tactic terms, and the reference ADRs rarely use that vocabulary. A metric based on similarity to the reference cannot distinguish "different from what this team wrote" from "worse", and so cannot credit the stages whose purpose is to change the decision's content: making trade-offs explicit, and checking coverage. Measuring those stages calls for instruments other than reference agreement, such as expert assessment of whether the decision addresses the stated quality concerns.

4.8. Earlier Development Runs

During development we ran two smaller evaluations, each with $N = 3$ items (seeds 42 and 43). Unlike the paired run, each system there re-ran Stages 1–3 independently. In the second of those runs, CADENCE-FULL scored above CADENCE-NC on 7 of 8 budget–metric comparisons, and we reported this as a positive signal for self-critique. The paired design shows that this was not a critique effect. Stage 4 never changes the decision text, so the difference came entirely from two independent samples of the same stochastic deliberation. This is why the paired design is necessary for ablating stochastic multi-stage pipelines. The other patterns from those runs do reproduce at $N = 30$: zero feasible items at $B = 5$, deliberation-based systems far below the single-pass baselines on ROUGE-1 and METEOR, and a narrower gap on BERTScore.

5. Discussion

5.1. Verification as a Detector: Where the Pipeline Loses Quality-Attribute Commitments

The central result is a mismatch between two stages. The quality-attribute advocates do raise their concerns, and Stage 3 checks precisely whether the decision addresses those concerns. Between the two, however, Stage 2's synthesis compresses the debate into a one-line decision, and the concerns of four of the five advocates rarely survive the compression (Section 4.5). Stage 3 then reports this correctly on all 30 items. Repairing the text afterwards, with the same 1.5B model, recovers coverage only partially: 29 of 30 decisions remain infeasible at $B = 5$.

This locates the problem at the interface between language and formal model, not in the solver. The architecture-optimization literature assumes a design space that experts specify in a formal model (Aleti et al., 2013; Martens et al., 2010; Koziolek et al., 2011). CADENCE instead extracts the design space from generated text, and the extraction is only as rich as the text. Three design changes follow from the data, and each can be tested with the same paired protocol:

1. *Make the decision structured.* Instead of matching tactic names in free text, the synthesis step could emit an explicit commitment per attribute, for example "security: least-privilege service boundaries". Constrained

decoding could keep it inside the catalog. Coverage then becomes a property of the output format, and the solver's job reduces to checking budget and trade-offs.

2. *Repair at the deliberation, not after it.* The uncovered attributes reported by Stage 3 could be returned to the corresponding advocate agents for another round, rather than to a stand-alone repair prompt that sees only the synthesized decision.

3. *Let the solver choose and the LLM explain.* With a given catalog and the stated requirements, the solver can select a feasible, minimal-trade-off tactic set on its own. Deliberation would then argue about which feasible set fits the context and write the rationale. This reverses the roles, and moves CADENCE closer to search-based design-space exploration with an LLM front end.

Each change also benefits from a richer, empirically grounded tactic model. The 26-tactic catalog, with one supported attribute per tactic, could be checked and extended against mined tactic–quality-attribute knowledge (Bi et al., 2021) and tactic–pattern interaction models (Harrison and Avgeriou, 2010; Márquez et al., 2023).

5.2. Implications for Practitioners

For teams that already use LLMs to draft ADRs, the results support two practical steps. First, a draft that reads well does not necessarily address the team's quality concerns. In our sample, the deliberated drafts left on average more than four of five required attributes without a named tactic. Second, the check that exposes this is cheap. Stage 3's tactic extraction is deterministic, the solver instance is tiny, and neither needs a large model. A coverage report that names which required attributes a draft leaves unaddressed is a useful review aid even when no automatic repair follows. Stage 4's residual weaknesses, rendered as a Consequences section, are a second inexpensive addition. They make the draft agree significantly more with what architects recorded, on three of the four metrics (Section 4.4). The final decision remains a human responsibility. The influence factors that dominate real decisions, such as team expertise, organizational constraints and existing commitments (Weinreich et al., 2015), are not in the title-only task context CADENCE receives here.

5.3. Implications for Research on LLM-Assisted Architecture

Two methodological lessons apply beyond CADENCE.

The first concerns *reference-based metrics*. They measure imitation of one recorded decision. The length-controlled analysis (Section 4.7) shows that a system can trail on every such metric for reasons other than output length, simply because it writes in a different vocabulary than the recorded ADR. This matches findings for code summarization and code generation (Roy et al., 2021; Evtikhiev et al., 2023; Haque et al., 2022). Small metric differences between LLM-for-ADR systems should therefore not be read as differences in decision quality. Stages that are meant to change content, such as trade-off reasoning and coverage checking, need instruments that measure that content: expert ratings against the stated quality concerns, or task-based evaluation with practitioners. LLM judges are a tempting substitute. Their agreement with human raters in software-engineering tasks varies, however, and same-family judging adds self-preference bias (Wang et al., 2025; He et al., 2026). Stage 4's critique is itself an LLM judge of this kind, which is why we do not report its scores as evidence of quality.

The second concerns *ablating stochastic pipelines*. When each system variant re-samples the shared upstream stages, differences between variants mix stage effects with sampling noise. On three items this noise was large enough to look like a consistent critique effect (Section 4.8). Sharing upstream outputs across variants, and testing paired differences, is a simple remedy, and we recommend it for any multi-stage LLM evaluation.

5.4. Threats to Validity

Construct validity. Agreement with one human-authored ADR is a proxy for decision quality. A decision can differ from the recorded one and still be sound, and the recorded one may itself be poor. We address this by reporting feasibility separately and by analyzing the metrics directly (RQ4), but we collected no expert judgments. Stage 3's coverage measure depends on matching tactic names in text. A decision that addresses an attribute in words not in the catalog is counted as not covering it. Some of the low coverage may be due to this, although the agents' prompts list the exact tactic names.

Internal validity. All systems share one backbone, decoding configuration and prompt set (Appendix B). Results may depend on these choices, in

particular on the 1.5B backbone, whose limited instruction following plausibly contributes to the compression problem. Generation is sampled, and each item ran once. The paired design removes sampling variance from the comparisons between Stage 2 and later stages, but not from the comparisons among the first three systems, which use independent calls.

External validity. The items come from open-source projects that publish ADRs, and the task gives only the ADR's title as context. Decisions in industrial settings, with richer context and different documentation practices, may behave differently. The five quality attributes and 26 tactics are a deliberately small subset of published catalogs (Márquez et al., 2023).

Conclusion validity. With $N = 30$, small effects such as retrieval's +0.004 BERTScore are detectable only because pairing removes between-item variance. We adjust for multiple comparisons within each metric. The feasibility result (1 of 30 at $B = 5$) is clear-cut. A rate close to zero is consistent with these data, but a moderate rate under a stronger backbone is not ruled out.

5.5. Engineering Lessons for Solver-Verified LLM Pipelines

Two implementation findings generalize. First, the lexicographic encoding in Stage 3 (Section 3.4) is required whenever callers can set weights on a secondary objective: under a weighted sum, a large enough trade-off weight silently overrides a requirement the caller believed was hard. Second, when a deterministic component consumes structured LLM output, a tolerant parser alone does not converge. Models produce novel format deviations that no regular expression anticipates. A bounded retry of the same prompt (Section 3.6) avoids these deviations by exploiting sampling variability, without enumerating them in advance.

6. Conclusion

We presented CADENCE, an approach to LLM-assisted architectural decision-making that combines precedent retrieval from mined ADRs, deliberation among quality-attribute advocates grounded in a tactic knowledge graph, verification of the decision's tactic commitments with an SMT solver, and a scored critique. Its aim is to join two lines of software architecture research that have developed separately: LLM-based ADR generation, which produces fluent decisions without checking them, and tactic-based and

search-based architecture design, which checks decisions but needs a formal model specified by hand.

A paired evaluation on 30 held-out ADRs from open-source projects gives a clear picture for a small local backbone. Retrieval helps slightly. Multi-agent deliberation lowers agreement with the recorded decisions, by more than output length explains. Solver verification is a reliable detector: it reports, correctly, that all 30 deliberated decisions leave required quality attributes without a committed tactic. Repair improves coverage but makes only one decision feasible, and budget slack does not help, because the binding constraint is the diversity of the committed tactics, not their number. Stage 4 does not change the decision text. Its residual weaknesses, rendered as consequences, do bring the text closer to what architects wrote. The paired design also corrected an earlier small-sample finding that had credited the critique stage with gains caused by sampling noise.

Future work follows from these findings: structured, catalog-constrained decision output; feedback of uncovered attributes to the advocate agents; solver-first selection of tactic sets; an empirically grounded tactic model; stronger backbones; and evaluation by architects of whether decisions address their quality concerns, rather than by agreement with a single recorded text.

Data and Code Availability

This paper is self-contained: the tactic catalog and every prompt are given in Appendix A and Appendix B, and all reported numbers appear in the tables. The implementation, the processed corpus, the evaluation scripts, the per-item outputs and solver diagnostics of the paired evaluation, and the statistical analysis are available at:

`https://github.com/vinhqdang/`
`Large-Language-Models-in-Service-Oriented-Ecosystems-Design`

The underlying ADR corpus is the public "Context Matters" replication package (Gupta et al., 2026a).

Acknowledgments

The author thanks the maintainers of the "Context Matters" ADR replication package for making a real, richly annotated ADR corpus publicly available, without which this work's retrieval stage and evaluation would not have been possible.

Appendix A. Tactic catalog

Table A.10 lists all 26 tactics of the knowledge graph G. Each tactic has one *supports* edge, to the quality attribute of its group, and the *trade-off* edges shown in the last column, each with the reason it burdens the named attribute. Trade-off edges are the only cross-attribute relations in G; Stage 3 minimizes their weighted cost and Stage 4 deducts a penalty for each one that lands on an attribute. The catalog is hand-curated in original wording from widely documented architectural concepts, not mined from the ADR corpus.

Table A.10: Full tactic catalog: effect and trade-off edges (target attribute and reason) for each of the 26 tactics.

Tactic	Effect	Trade-off edges
Performance		
Caching	Store frequently-accessed results to avoid recomputation or repeated I/O.	*maintainability*: cache invalidation and staleness become an ongoing concern
Connection pooling	Reuse expensive connections (DB, network) instead of opening one per request.	*maintainability*: pool sizing and lifecycle tuning adds operational complexity
Asynchronous processing via message queues	Decouple slow work from the request path via a queue and background workers.	*maintainability*: harder to trace an end-to-end request across queue boundaries; *security*: the queue and its consumers become additional attack surface
Data denormalization	Duplicate data to avoid expensive joins or cross-service calls at read time.	*maintainability*: duplicated data must be kept consistent across writers
Read-through/write-behind data access	Batch or defer writes and prefetch reads to reduce per-operation latency.	*maintainability*: failure windows can lose or reorder deferred writes
Security		
Authentication	Verify the identity of a caller before granting access.	*performance*: adds per-request latency for credential verification; *cost/operability*: requires identity infrastructure to run and maintain

Tactic	Effect	Trade-off edges
Fine-grained authorization / access control	Restrict operations to callers with the specific permissions they need.	*maintainability*: permission policies must be kept in sync with the system's evolution
Encryption in transit and at rest	Protect data from interception or unauthorized disk-level access.	*performance*: adds CPU overhead for encrypt/decrypt operations; *cost/operability*: key management and rotation is an ongoing operational task
Rate limiting / throttling	Cap how much load a single caller can place on the system.	*performance*: adds bookkeeping overhead to every request, including legitimate ones
Least-privilege service boundaries	Give each service only the access it strictly needs, isolating blast radius.	*maintainability*: more, narrower services means more boundaries to coordinate changes across
Input validation at trust boundaries	Reject malformed or malicious input as early as possible.	*performance*: adds validation overhead on every incoming request
Maintainability		
Modular decomposition by business capability	Split the system along business boundaries so each module changes independently.	*performance*: cross-module calls add latency that a single module wouldn't have; *cost/operability*: more independently deployable units means more to operate
Versioned, well-defined API contracts	Make module/service boundaries explicit and evolve them without breaking consumers.	*cost/operability*: contract review and versioning tooling adds process overhead
Automated regression test suite	Catch behavioral regressions automatically before they reach production.	*cost/operability*: CI compute time and suite upkeep is a recurring cost
Centralized structured logging and observability	Make system behavior inspectable across components for debugging and audits.	*cost/operability*: log storage and processing infrastructure has an ongoing cost; *security*: logs can leak sensitive data if not deliberately scrubbed
Dependency injection / inversion of control	Decouple components from concrete implementations of their dependencies.	*performance*: the extra indirection has a small but nonzero runtime cost
Scalability		

Tactic	Effect	Trade-off edges
Horizontal scaling with stateless instances	Add more identical instances behind a load balancer instead of growing one instance.	*cost/operability*: more running instances means more infrastructure to pay for and operate; *maintainability*: session/request state must be externalized, not held in-process
Database sharding	Partition data across multiple database instances by a shard key.	*maintainability*: cross-shard queries and rebalancing become genuinely hard problems; *cost/operability*: operating multiple database instances multiplies operational surface
Event-driven architecture	Components communicate via events rather than direct synchronous calls.	*maintainability*: reasoning about system state requires tracing asynchronous event flows; *security*: each event channel is an additional integration point to secure
Auto-scaling infrastructure	Automatically add or remove capacity in response to observed load.	*performance*: newly-started instances suffer cold-start latency during scale-up
Read replicas	Serve read traffic from replicated copies of the primary data store.	*maintainability*: callers must tolerate and reason about replication lag
Cost/operability		
Managed cloud services over self-hosted infrastructure	Use a vendor-operated service instead of running and patching the equivalent yourself.	*security*: trust and blast radius now partly depend on the vendor's shared-responsibility model; *maintainability*: the system becomes coupled to that vendor's specific service semantics
Infrastructure as code	Define infrastructure declaratively so environments are reproducible and cheap to recreate.	none
Multi-tenancy	Share infrastructure across multiple customers/tenants instead of provisioning per-tenant.	*security*: a boundary failure between tenants has a much larger blast radius; *performance*: a noisy tenant can degrade performance for others sharing the same resources

Tactic	Effect	Trade-off edges
Serverless / function-as-a-service deployment	Pay only for actual invocation time instead of provisioning always-on capacity.	*performance*: cold starts add latency to infrequently-invoked functions; *maintainability*: execution model constraints (timeouts, statelessness) shape how code must be written
Circuit breaker for downstream dependencies	Stop calling a failing dependency instead of paying for retries against it.	*maintainability*: failure-handling logic and fallback paths add real code complexity

Appendix B. Prompt templates

All prompts used by the pipeline and baselines are reproduced below; angle brackets mark values filled in at run time. Agents (Stage 2) receive the system prompt in Section Appendix B.1 with every call; all other calls have no system prompt. $\langle a \rangle$ is a quality attribute, $\langle c \rangle$ the decision context, and precedent lines are the titles of the k retrieved ADRs.

Appendix B.1. Stage 2: quality-attribute agent

```
[system]
You are the <a> advocate in an architectural decision-making deliberation. Argue for the
     decision that best serves <a>, while acknowledging real trade-offs.
Tactics you can draw on for <a>: <names of the tactics supporting <a>, comma-separated>

[round 1, propose]
Decision context:
<c>

Precedent decisions from similar past projects:
- <title of precedent 1>
- ...

Propose and justify a position from the <a> perspective for this decision.

[rounds 2..R, critique]
Current candidate decision:
<all agents' latest positions, one per line>

Other advocates' positions this round:
- [<attribute>] <position>
- ...

From the <a> perspective, critique or refine this candidate, citing specific trade-offs
     where relevant.
```

Appendix B.2. Stage 2: synthesis

```
Decision context:
<c>

Full deliberation transcript:
[<attribute> r<round>] <position>
...

Synthesize the deliberation into a final decision. Respond in exactly this format:
CANDIDATE: <one or two sentence decision>
RATIONALE: <one paragraph rationale>
```

Appendix B.3. Stage 3: repair

```
The following architectural decision candidate cannot be verified as feasible:
<current candidate>

Constraint: the decision may commit to at most <B> distinct architectural tactics (an
     operability budget).
Required quality attributes that must each be addressed by at least one tactic: <Q,
     comma-separated>.
The best attempt so far leaves these quality attributes unaddressed: <uncovered
     attributes> (at least <unsat-core attributes> are known to conflict with the budget).

Tactics you could name explicitly to address each unaddressed attribute:
- <uncovered attribute>: <names of the tactics supporting it>
- ...

Revise the decision to fit within the tactic budget while covering as many of the required
     quality attributes as possible -- naming specific tactics from the lists above by name
     where you use them -- consolidating around fewer, higher-impact tactics where needed.
     Respond in exactly this format:
CANDIDATE: <one or two sentence decision>
RATIONALE: <one paragraph rationale>
```

Appendix B.4. Stage 4: qualitative critique

```
Decision: <decision>
Rationale: <rationale>

Critique this architectural decision from each quality attribute perspective below. Score
     each 0 (fails this attribute) to 10 (exemplary), and name one specific residual
     weakness if any exist (or 'none'). Respond in exactly this format, one block per
     attribute:
<ATTRIBUTE>_SCORE: <0-10>
<ATTRIBUTE>_WEAKNESS: <specific weakness, or 'none'>
... (one block for each required attribute)
```

Appendix B.5. Baselines

```
[Zero-Shot]
Decision context:
<c>

Write an architectural decision record addressing this context.

[Retrieval-Only]
Decision context:
<c>

Precedent decisions from similar past projects:
- <title of precedent 1>
- ...

Write an architectural decision record addressing this context, informed by the precedents
     above.
```

References

Ahmad, A., Waseem, M., Liang, P., Fahmideh, M., Aktar, M.S., Mikkonen, T., 2023. Towards human-bot collaborative software architecting with ChatGPT, in: Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering (EASE), ACM. pp. 279–285. doi:10.1145/3593434.3593468.

Al-Naeem, T., Gorton, I., Babar, M.A., Rabhi, F., Benatallah, B., 2005. A quality-driven systematic approach for architecting distributed software applications, in: Proceedings of the 27th International Conference on Software Engineering (ICSE 2005), ACM. pp. 244–253. doi:10.1145/1062455.1062508.

Aleti, A., Bjornander, S., Grunske, L., Meedeniya, I., 2009. ArcheOpterix: An extendable tool for architecture optimization of AADL models, in: 2009 ICSE Workshop on Model-Based Methodologies for Pervasive and Embedded Software (MOMPES), IEEE. pp. 61–71. doi:10.1109/MOMPES.2009.5069138.

Aleti, A., Buhnova, B., Grunske, L., Koziolek, A., Meedeniya, I., 2013. Software architecture optimization methods: A systematic literature review. IEEE Transactions on Software Engineering 39, 658–683. doi:10.1109/TSE.2012.64.

Bachmann, F., Bass, L., Klein, M., 2003. Deriving Architectural Tactics: A Step Toward Methodical Architectural Design. Technical Report

CMU/SEI-2003-TR-004. Software Engineering Institute, Carnegie Mellon University. doi:10.21236/ADA413701.

Bagheri, H., Sullivan, K., 2016. Model-driven synthesis of formally precise, stylized software architectures. Formal Aspects of Computing 28, 441–467. doi:10.1007/s00165-016-0360-8.

Bass, L., Clements, P., Kazman, R., 2021. Software Architecture in Practice. 4th ed., Addison-Wesley Professional, Boston, MA, USA. ISBN: 978-0-13-688609-9.

Benavides, D., Segura, S., Ruiz-Cortés, A., 2010. Automated analysis of feature models 20 years later: A literature review. Information Systems 35, 615–636. doi:10.1016/j.is.2010.01.001.

Bi, T., Liang, P., Tang, A., Xia, X., 2021. Mining architecture tactics and quality attributes knowledge in Stack Overflow. Journal of Systems and Software 180, 111005. doi:10.1016/j.jss.2021.111005.

Borowa, K., Zalewski, A., Kijas, S., 2021. The influence of cognitive biases on architectural technical debt, in: 2021 IEEE 18th International Conference on Software Architecture (ICSA), IEEE. pp. 115–125. doi:10.1109/ICSA51549.2021.00019.

Buchgeher, G., Schöberl, R., Geist, P., Dorninger, J., Haindl, C., Weinreich, R., 2023. Using architecture decision records in open source projects – an MSR study on GitHub. IEEE Access 11, 63725–63740. doi:10.1109/ACCESS.2023.3287654.

Capilla, R., Diaz-Pace, J.A., Ramírez, Y., Pérez, J., Rodríguez-Horcajo, V., 2025. Towards supporting quality architecture evaluation with LLM tools. arXiv:2506.00150.

Capilla, R., Diaz-Pace, J.A., Ramírez, Y., Pérez, J., Rodríguez-Horcajo, V., 2026. Supporting architecture evaluation for ATAM scenarios with LLMs. arXiv:2603.28914.

Capilla, R., Jansen, A., Tang, A., Avgeriou, P., Babar, M.A., 2016. 10 years of software architecture knowledge management: Practice and future. Journal of Systems and Software 116, 191–205. doi:10.1016/j.jss.2015.08.054.

Chan, C.M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., Liu, Z., 2024. ChatEval: Towards better LLM-based evaluators through multi-agent debate, in: Proc. Int. Conf. Learning Representations (ICLR). ArXiv:2308.07201.

Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., et al., 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv preprint. ArXiv:2507.06261.

Cortellessa, V., Di Pompeo, D., Stoico, V., Tucci, M., 2023. Many-objective optimization of non-functional attributes based on refactoring of software models. Information and Software Technology 157, 107159. doi:10.1016/j.infsof.2023.107159.

Dhar, R., Kakran, A., Karan, A., Vaidhyanathan, K., Varma, V., 2025. DRAFT-ing architectural design decisions using LLMs. arXiv preprint. ArXiv:2504.08207.

Dhar, R., Vaidhyanathan, K., Varma, V., 2024. Can LLMs generate architectural design decisions? – an exploratory empirical study, in: Proc. IEEE 21st Int. Conf. Software Architecture (ICSA). ArXiv:2403.01709.

Dhar, R., Vaidhyanathan, K., Varma, V., 2026. AgenticAKM: Enroute to agentic architecture knowledge management. arXiv:2602.04445.

Diaz-Pace, J.A., Tommasel, A., Capilla, R., Ramírez, Y.E., 2025. Architecture exploration and reflection meet LLM-based agents, in: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), IEEE. pp. 1–5. doi:10.1109/ICSA-C65153.2025.00015.

Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I., 2024. Improving factuality and reasoning in language models through multiagent debate, in: Proc. 41st Int. Conf. Machine Learning (ICML), pp. 11733–11763. ArXiv:2305.14325.

Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., Taibi, D., 2026. Generative AI for software architecture. applications, challenges, and future directions. Journal of Systems and Software 231, 112607. doi:10.1016/j.jss.2025.112607.

Evtikhiev, M., Bogomolov, E., Sokolov, Y., Bryksin, T., 2023. Out of the BLEU: How should we assess quality of the code generation models? Journal of Systems and Software 203, 111741. doi:10.1016/j.jss.2023.111741.

Falessi, D., Cantone, G., Kazman, R., Kruchten, P., 2011. Decision-making techniques for software architecture design: A comparative survey. ACM Computing Surveys 43, 1–28. doi:10.1145/1978802.1978812.

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M., 2023. Large language models for software engineering: Survey and open problems, in: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), IEEE. pp. 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008.

Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., et al., 2025. Gemma 3 technical report. arXiv preprint. ArXiv:2503.19786.

Gupta, A., Dhar, R., Feitosa, D., Vaidhyanathan, K., 2026a. Context matters – replication package. Zenodo. doi:10.5281/zenodo.18370195.

Gupta, A., Dhar, R., Feitosa, D., Vaidhyanathan, K., 2026b. Context matters: Evaluating context strategies for automated ADR generation using LLMs, in: Proc. EASE 2026 Research Track. Accepted. arXiv:2604.03826.

Hao, Y., Chen, Y., Zhang, Y., Fan, C., 2024. Large language models can solve real-world planning rigorously with formal verification tools. arXiv preprint. ArXiv:2404.11891.

Haque, S., Eberhart, Z., Bansal, A., McMillan, C., 2022. Semantic similarity metrics for evaluating source code summarization, in: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension (ICPC), ACM. pp. 36–47. doi:10.1145/3524610.3527909.

Harrison, N.B., Avgeriou, P., 2010. How do architecture patterns and tactics interact? A model and annotation. Journal of Systems and Software 83, 1735–1758. doi:10.1016/j.jss.2010.04.067.

Harrison, N.B., Avgeriou, P., Zdun, U., 2007. Using patterns to capture architectural decisions. IEEE Software 24, 38–45. doi:10.1109/MS.2007.124.

Hatalis, K., Christou, D., Kondapalli, V., 2025. Review of case-based reasoning for LLM agents: Theoretical foundations, architectural components, and cognitive integration. arXiv preprint. ArXiv:2504.06943.

He, J., Shi, J., Zhuo, T.Y., Treude, C., Sun, J., Du, X., Xing, Z., Lo, D., 2026. LLM-as-a-judge for software engineering: Literature review, vision, and the road ahead. ACM Transactions on Software Engineering and Methodology 35, 1–30. doi:10.1145/3797276.

He, J., Treude, C., Lo, D., 2025. LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. ACM Transactions on Software Engineering and Methodology 34, 1–30. doi:10.1145/3712003.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology 33, 1–79. doi:10.1145/3695988.

Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D., 2024. Large language models cannot self-correct reasoning yet, in: Proc. Int. Conf. Learning Representations (ICLR). ArXiv:2310.01798.

Indykov, V., Strüber, D., Wohlrab, R., 2025. Architectural tactics to achieve quality attributes of machine-learning-enabled systems: A systematic literature review. Journal of Systems and Software 223, 112373. doi:10.1016/j.jss.2025.112373.

ISO/IEC, 2023. ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model. Technical Report. International Organization for Standardization. Geneva, Switzerland.

Jahić, J., Sami, A., 2024. State of practice: LLMs in software engineering and software architecture, in: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), IEEE. pp. 311–318. doi:10.1109/ICSA-C63560.2024.00059.

Jansen, A., Bosch, J., 2005. Software architecture as a set of architectural design decisions, in: 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05), IEEE. pp. 109–120. doi:10.1109/WICSA.2005.61.

Kazman, R., Asundi, J., Klein, M., 2001. Quantifying the costs and benefits of architectural decisions, in: Proceedings of the 23rd International Conference on Software Engineering (ICSE 2001), IEEE. pp. 297–306. doi:10.1109/ICSE.2001.919103.

Kazman, R., Klein, M., Barbacci, M., Longstaff, T., Lipson, H., Carriere, J., 1998. The architecture tradeoff analysis method, in: Proceedings of the Fourth IEEE International Conference on Engineering of Complex Computer Systems (ICECCS), IEEE. pp. 68–78. doi:10.1109/ICECCS.1998.706657.

Koziolek, A., Koziolek, H., Reussner, R., 2011. PerOpteryx: Automated application of tactics in multi-objective software architecture optimization, in: Proceedings of the Joint ACM SIGSOFT Conference QoSA and ACM SIGSOFT Symposium ISARCS, ACM. pp. 33–42. doi:10.1145/2000259.2000267.

Kruchten, P., 2004. An ontology of architectural design decisions in software intensive systems, in: Bosch, J. (Ed.), Proceedings of the 2nd Groningen Workshop on Software Variability Management, Groningen, The Netherlands. pp. 54–61.

Lewis, P., et al., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Proc. NeurIPS. ArXiv:2005.11401.

Li, R., Zhang, Y., Zhou, X., Liang, P., Sun, W., Xuan, J., Jin, Z., Liu, Y., 2025. MAAD: Automate software architecture design through knowledge-driven multi-agent collaboration. arXiv preprint. ArXiv:2507.21382.

Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Shi, S., Tu, Z., 2024. Encouraging divergent thinking in large language models through multi-agent debate, in: Proc. 2024 Conf. Empirical Methods in Natural Language Processing (EMNLP). ArXiv:2305.19118.

Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., Lou, Y., 2026. Large language model-based agents for software engineering: A survey. ACM Transactions on Software Engineering and Methodology doi:10.1145/3796507.

Madaan, A., et al., 2023. Self-refine: Iterative refinement with self-feedback, in: Proc. Advances in Neural Information Processing Systems (NeurIPS). ArXiv:2303.17651.

Márquez, G., Astudillo, H., Kazman, R., 2023. Architectural tactics in software architecture: A systematic mapping study. Journal of Systems and Software 197, 111558. doi:10.1016/j.jss.2022.111558.

Martens, A., Koziolek, H., Becker, S., Reussner, R., 2010. Automatically improve software architecture models for performance, reliability, and cost using evolutionary algorithms, in: Proceedings of the First Joint WOSP/SIPEW International Conference on Performance Engineering, ACM. pp. 105–116. doi:10.1145/1712605.1712624.

Mirakhorli, M., Cleland-Huang, J., 2016. Detecting, tracing, and monitoring architectural tactics in code. IEEE Transactions on Software Engineering 42, 205–220. doi:10.1109/TSE.2015.2479217.

Mirakhorli, M., Shin, Y., Cleland-Huang, J., Cinar, M., 2012. A tactic-centric approach for automating traceability of quality concerns, in: 2012 34th International Conference on Software Engineering (ICSE), IEEE. pp. 639–649. doi:10.1109/ICSE.2012.6227153.

Moore, M., Kazman, R., Klein, M., Asundi, J., 2003. Quantifying the value of architecture design decisions: Lessons from the field, in: Proceedings of the 25th International Conference on Software Engineering (ICSE 2003), IEEE. doi:10.1109/ICSE.2003.1201237.

Olausson, T.X., Gu, A., Lipkin, B., Zhang, C.E., Solar-Lezama, A., Tenenbaum, J.B., Levy, R., 2023. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers, in: Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 5153–5176. doi:10.18653/v1/2023.emnlp-main.313. arXiv:2310.15164.

Pan, L., Albalak, A., Wang, X., Wang, W.Y., 2023. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning, in: Findings of the Assoc. Comput. Linguistics: EMNLP 2023, pp. 3806–3824. doi:10.18653/v1/2023.findings-emnlp.248.

Razavian, M., Tang, A., Capilla, R., Lago, P., 2016. In two minds: How reflections influence software design thinking. Journal of Software: Evolution and Process 28, 394–426. doi:10.1002/smr.1776.

Roy, D., Fakhoury, S., Arnaoudova, V., 2021. Reassessing automatic evaluation metrics for code summarization tasks, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM. pp. 1105–1116. doi:10.1145/3468264.3468588.

Schmid, L., Hey, T., Armbruster, M., Corallo, S., Fuchß, D., Keim, J., Liu, H., Koziolek, A., 2025. Software architecture meets LLMs: A systematic literature review. arXiv:2505.16697.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Proc. NeurIPS. ArXiv:2303.11366.

Soliman, M., Keim, J., 2025. Do large language models contain software architectural knowledge? An exploratory case study with GPT, in: 2025 IEEE 22nd International Conference on Software Architecture (ICSA), IEEE. pp. 13–24. doi:10.1109/ICSA65012.2025.00012.

Su, R., Bakhtin, A., Ahmad, N., Esposito, M., Lenarduzzi, V., Taibi, D., 2026. Evaluating large language models for detecting architectural decision violations. arXiv preprint. ArXiv:2602.07609.

Szeider, S., 2025. Bridging language models and symbolic solvers via the model context protocol, in: Proc. 28th Int. Conf. Theory and Applications of Satisfiability Testing (SAT 2025). doi:10.4230/LIPIcs.SAT.2025.30.

Tang, A., Avgeriou, P., Jansen, A., Capilla, R., Babar, M.A., 2010. A comparative study of architecture knowledge management tools. Journal of Systems and Software 83, 352–370. doi:10.1016/j.jss.2009.08.032.

Tang, A., Babar, M.A., Gorton, I., Han, J., 2006. A survey of architecture design rationale. Journal of Systems and Software 79, 1792–1804. doi:10.1016/j.jss.2006.04.029.

Tyree, J., Akerman, A., 2005. Architecture decisions: Demystifying architecture. IEEE Software 22, 19–27. doi:10.1109/MS.2005.27.

van Vliet, H., Tang, A., 2016. Decision making in software architecture. Journal of Systems and Software 117, 638–644. doi:10.1016/j.jss.2016.01.017.

Wang, R., Guo, J., Gao, C., Fan, G., Chong, C.Y., Xia, X., 2025. Can LLMs replace human evaluators? An empirical study of LLM-as-a-judge in software engineering. Proceedings of the ACM on Software Engineering 2, 1955–1977. doi:10.1145/3728963.

Weinreich, R., Groher, I., Miesbauer, C., 2015. An expert survey on kinds, influence factors and documentation of design decisions in practice. Future Generation Computer Systems 47, 145–160. doi:10.1016/j.future.2014.12.002.

Wiratunga, N., Abeyratne, R., Jayawardena, L., Martin, K., Massie, S., Nkisi-Orji, I., Weerasinghe, R., Liret, A., Fleisch, B., 2024. CBR-RAG: Case-based reasoning for retrieval augmented generation in LLMs for legal question answering, in: Proc. Int. Conf. Case-Based Reasoning (ICCBR). ArXiv:2404.04302.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., et al., 2025. Qwen3 technical report. arXiv preprint. (Qwen Team). arXiv:2505.09388.

Zalewski, A., Borowa, K., Ratkowski, A., 2017. On cognitive biases in architecture decision making, in: Software Architecture (ECSA 2017), Springer. pp. 123–137. doi:10.1007/978-3-319-65831-5_9.

Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., et al., 2025. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. arXiv preprint. ArXiv:2508.06471.

Zhang, Y., Li, R., Liang, P., Sun, W., Liu, Y., 2025. Knowledge-based multi-agent framework for automated software architecture design, in: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion), ACM. pp. 530–534. doi:10.1145/3696630.3728493.

Zhou, X., Li, R., Liang, P., Zhang, B., Shahin, M., Li, Z., Yang, C., 2025. Using LLMs in generating design rationale for software architecture decisions. ACM Trans. Softw. Eng. Methodol. (TOSEM) Accepted for publication. arXiv:2504.20781.

Zimmermann, O., Koehler, J., Leymann, F., Polley, R., Schuster, N., 2009. Managing architectural decision models with dependency relations, integrity constraints, and production rules. Journal of Systems and Software 82, 1249–1267. doi:10.1016/j.jss.2009.01.039.